
NI-DAQmx User Manual

2026-06-25



Contents

Tasks in NI-DAQmx	3
Creating Tasks with the API	4
Using the Start Task function/VI	5
Aborting a Task	7
Using Is Task Done	8
Using Wait Until Done	8
When Is A Task Done?	9
Task State Model	9
Unverified State	10
Verified State	10
Reserved State	11
Committed State	11
Running State	12
Running to Committed State	12
Committed to Verified State	12
Explicit Versus Implicit State Transitions	13
Implicit Task State Transitions	14
Task Moves Through Multiple States at the Same Time	14
Operations That Require State Transitions	15
Transitioning the State Backwards	15

Tasks in NI-DAQmx

A task is a collection of one or more virtual channels with timing, triggering, and other properties. Conceptually, a task represents a measurement or generation you want to perform. All channels in a task must be of the same I/O type, such as analog input or counter output. However, a task can include channels of different measurement types, such as an analog input temperature channel and an analog input voltage channel. For most devices, only one task per subsystem can run at once, but some devices can run multiple tasks simultaneously. With some devices, you can include channels from multiple devices in a task. To perform a measurement or a generation with a task, follow these steps:

1. Create or load a task. You can create tasks interactively with the DAQ Assistant or programmatically in your ADE such as LabVIEW or LabWindows/CVI.
2. Configure the channel, timing, and triggering properties as necessary.
3. Optionally, perform various task state transitions to prepare the task to perform the specified operation.
4. Read or write samples.
5. Clear the task.

If appropriate for your application, repeat steps 2 through 4. For instance, after reading or writing samples, you can reconfigure the virtual channel, timing, or triggering properties and then read or write additional samples based on this new configuration.

If properties need to be set to values other than their defaults for your task to be successful, your program must set these properties every time it executes. For example, if you run a program that sets property A to a nondefault value and follow that with a second program that does not set property A, the second program uses the default value of property A. The only way to avoid setting properties programmatically each time a program runs is to use virtual channels and/or tasks created in the DAQ Assistant.

Related concepts:

- [Simultaneous Tasks](#)
- [Multidevice Tasks](#)

- [Task State Model](#)
- [Creating Channels and Tasks with the DAQ Assistant](#)

Related tasks:

- [Creating Tasks with the API](#)

Creating Tasks with the API

The following example illustrates how to create a task with the API:

Problem

Create an NI-DAQmx task to measure temperature in the range 50°C to 200°C using a J-type thermocouple that is wired to channel 0 on an M Series device configured as Device 1. Sample the temperature 10 times per second, and acquire 10,000 samples. Use LabVIEW or LabWindows/CVI to write your application.

Solution

1. Call the AI Temp TC instance of the DAQmx Create Virtual Channel VI in LabVIEW (DAQmxCreateAIThrmcplChan function in LabWindows/CVI).
2. Specify `Dev1/ai0` as the physical channel for the device connected to the thermocouple signal.
3. Specify `myThermocoupleChannel` as the name to assign to your virtual channel.
4. Select the appropriate values for the thermocouple type and range inputs. NI-DAQmx applies these attributes to the virtual channel.
5. Call the Sample Clock instance of DAQmx Timing VI in LabVIEW (or DAQmxCfgSampClkTiming function in LabWindows/CVI), specifying a `rate` of 10 Hz and a `sample mode` of finite.
6. Call the DAQmx Start Task VI (DAQmxStartTask in LabWindows/CVI).
7. Call the Analog 1D DBL 1Chan NSamp instance of DAQmx Read VI (DAQmxReadAnalogF64 in LabWindows/CVI), specifying number of `samples per channel` as 10,000.
8. Call the DAQmx Stop Task VI (DAQmxStopTask function in LabWindows/CVI) after the desired number of samples have been acquired.

9. Call the DAQmx Clear Task VI (DAQmxClearTask function in LabWindows/CVI).

You have now created a task called myTemperatureTask that uses a local virtual channel called myThermocoupleChannel.



Note You also can use the DAQ Assistant to create the same task and generate the code to run the task.

Related concepts:

- [Creating Channels and Tasks with the DAQ Assistant](#)
- [Choosing Whether to Use the API or the DAQ Assistant](#)

Using the Start Task function/VI

To explicitly start a task, call the Start Task function/VI. You auto-start a task when you perform some other operation that implicitly starts the task. For instance, calling a Read function/VI or a Write function/VI might implicitly start the task if one is not already started. How to specify this behavior depends on the operation that your task performs. By default, the Read function/VI and the Write function/VI for a single sample automatically starts a task.

Related concepts:

- [Committed State](#)
- [Running State](#)
- [Task State Model](#)
- [Verified State](#)

Starting a Finite Measurement Task

If you have specified a task to perform a finite measurement, you do not need to call the Start Task function/VI, nor do you need to change the default behavior of the DAQmx Read function/VI. Calling the Read function/VI starts your task, performs the finite measurement, and stops the task after the last sample is read. The task returns to its state before you called the read operation. However, if you need to perform additional read operations after the task has been stopped (in other words, if you want

to read earlier locations in the buffer), the default behavior is insufficient for two reasons:

1. The task is returned to the Verified state and the samples are no longer accessible.
2. Future calls of the Read function/VI start new read operations rather than reading from the completed operation.

For this situation, explicitly commit the task by calling the Control Task function/VI with the `Action` parameter set to Commit. Then, after performing the initial read operation and before performing the subsequent read operations, set the Auto-Start Read attribute/property to False.

Starting a Continuous Measurement Task

For a continuous measurement, explicitly call the Start Task function/VI, perform the desired read operations, and call the Stop Task function/VI to stop the continuous measurement. When you perform a read operation in a loop—regardless if the read operation performs a single-sample, on-demand read, or a multiple-sample, hardware-timed read—call the Start Task function/VI before entering the loop and call the Stop Task function/VI after leaving the loop.

Starting an Analog Output Task

The behavior of the Write function/VI is more complicated. Calling the Write function/VI always results in the task transitioning to at least the Committed state. Whether the task transitions to the Running state depends on the value of the `Auto-Start` parameter.

For single-sample write operation, call a single-sample version of the Write function/VI. This call implicitly starts the task, writes the single sample, and stops the task. For a multiple-sample, on-demand write operation, call the Write function/VI, but also set the `Auto-Start` parameter to True, which by default is set to False. This call implicitly starts the task, writes the multiple samples, and stops the task.

For a multiple-sample, hardware-timed write operation, first call the Write function/VI to write the samples to generate, explicitly call the Start Task function/VI, wait for the samples to be generated by calling the Wait Until Done function/VI, and then explicitly call the Stop Task function/VI.

If you attempt to perform a hardware-timed generation with the `Auto-Start` parameter of the Write function/VI set to `True` either because you explicitly set it to `true` or because you are using a single-sample Write function/VI, the operation might fail because the samples that you write are not transferred to the device in time to generate the waveform. As a result, when performing hardware-timed generations, always write at least part of the waveform to generate before starting the task.

Improving Performance with the Start Task function/VI

There are other situations in which you should explicitly call the DAQmx Start Task function/VI and the DAQmx Stop Task function/VI, even though you are not required to do so. When you call the Read function/VI or the Write function/VI in a loop, you can significantly improve performance if you explicitly call the Start Task function/VI before entering the loop and call the Stop Task function/VI after exiting the loop. Without explicitly calling the Start Task function/VI before entering the loop, the task must implicitly transition from its current state to the Running state before performing the read or write operation. After the read or write operation is complete, the task must implicitly transition from the Running state back to its previous state. These implicit state transitions occur for every iteration of the loop, which is inefficient.

Aborting a Task

Several conditions cause a task to abort:

- To explicitly abort a task, call the DAQmx Control Task function/VI with the `Action` parameter set to `Abort`. In general, aborting a task is not a normal operation. It is intended for exceptional situations.
- In LabVIEW, you also can abort a task by clicking the **Abort Execution** button. Doing so results in all tasks created in that VI hierarchy to be aborted and then cleared.
- If you remove a device from the system, all tasks currently using the resources of that device are aborted.
- If you call the DAQmx Reset Device function/VI to restore a device to its initial configuration, all tasks currently utilizing the resources of that device are aborted.

When a task is aborted, it is returned to the Verified state. If the task is running, it is stopped as soon as possible and is then unreserved. After a task has been aborted, you

can continue to use the task. However, you might need to transition the task back to its previous state before continuing the specified operation.

Related concepts:

- [Verified State](#)

Using Is Task Done

You can use the Is Task Done function/VI for applications in which you need to monitor the progress of a task running in one section of your application from another section of your application.

In general, use the Is Task Done function/VI with continuous measurements and generations when you are not actively reading or writing samples but want to monitor for errors.

Related concepts:

- [When Is A Task Done?](#)

Using Wait Until Done

You might need to call the Wait Until Done function/VI to ensure that the specified operation is complete before you stop the task.

The most common example is a finite generation. If you start a task that performs a finite generation and then immediately stop the task, the generation probably has not completed when you stop the task. As a result, the generation does not complete as expected. To ensure that the finite generation completes as expected, call the Wait Until Done function/VI before stopping the task. After the Wait Until Done function/VI executes, the finite generation has been completed, and you can stop the task.

In general, use the Wait Until Task Done function/VI with finite measurements and generations.

Related concepts:

- [When Is A Task Done?](#)

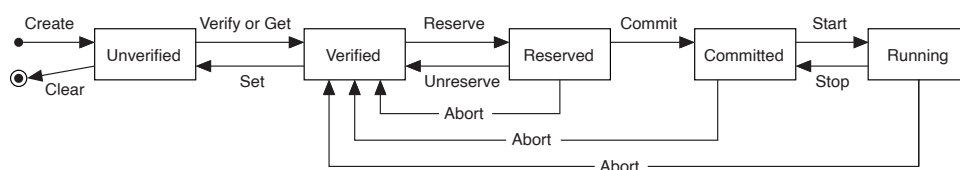
When Is A Task Done?

If the measurement or generation is finite, the task is done when you acquire or generate the final sample or when you call the Stop Task function/VI. If the measurement or generation is continuous (including on-demand timing), or if retriggering is enabled, the task is not done until you call the Stop Task function/VI. In addition, the task is done if a fatal error is generated while performing the measurement or generation, or you abort the measurement or generation. Check for errors and warnings to verify the task completed successfully.

Task State Model

NI-DAQmx uses a task state model to improve ease of use and speed up driver performance.

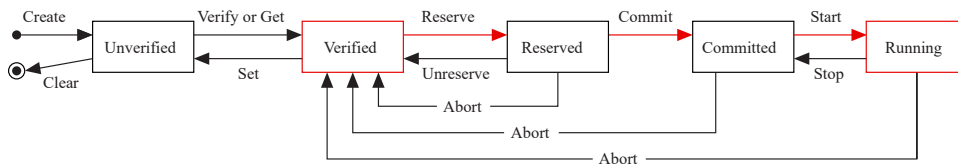
The task state model consists of five states—Unverified, Verified, Reserved, Committed, and Running. You call the Start Task function/VI, Stop Task function/VI, and Control Task function/VI to transition the task from one state to another. The task state model is very flexible. You can choose to interact with as little or as much of the task state model as your application requires.



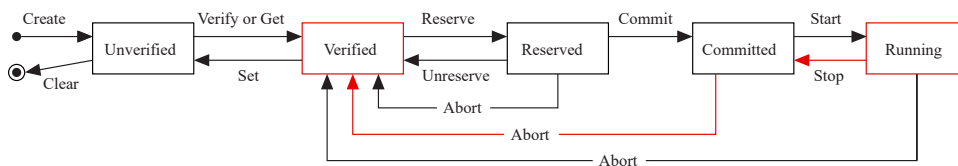
If you explicitly invoke a state transition that has already occurred, it is not repeated and an error is not returned. For example, if the task has already reserved its resources and, therefore, is in the Reserved state, calling the Control Task function/VI with the `Action` parameter set to Reserve does not reserve the resources again.

Sometimes, calling a function/VI may require multiple state transitions, such as calling the Start Task function/VI while in the Verified state. In these cases, the task will implicitly transition between each of the necessary states to get to the final desired

state, as shown in the following diagram.



Transitioning backwards in the Task State Model will undo any implicit forwards transitions in addition to the requested explicit transition. Continuing with the example above, calling the Stop Task function/VI after implicitly transitioning to the Running State from the Verified State will cause the task to return to the Verified State, as shown in the following diagram.



Unverified State

When a task is created or loaded, either explicitly or implicitly, it is in the Unverified state. In this state, you configure the timing, triggering, and channel attributes/properties of the task.

Verified State

NI-DAQmx checks the timing, triggering, and channel attributes/properties for correctness when the task transitions from the Unverified to the Verified state. You can explicitly perform this transition by calling the Control Task function/VI with `Action` set to `Verify`. While NI-DAQmx detects and verifies some invalid values for attributes/properties immediately when you set the attribute/property, NI-DAQmx cannot verify other values immediately because they depend on other attributes/properties and the devices being used. NI-DAQmx checks the value of these attributes/properties during the verify transition and reports any invalid values at that time. If NI-DAQmx finds no invalid values, the task is successfully verified and transitions to the Verified state. Otherwise, it remains in the Unverified state.

In certain cases, NI-DAQmx will coerce the values of attributes/properties when

successfully verifying a task rather than generating an error. This is done when the value set on the attribute/property cannot be met exactly as specified and coercing it to a legal value has little functional impact on the task.

Related concepts:

- [Coercion](#)

Reserved State

The resources a task uses to perform the specified operation are acquired exclusively when the task transitions from the Verified state to the Reserved state. These resources can be clocks or channels on a device, trigger lines on a PXI chassis, or buffer memory in the computer. Reserving these resources prevents other tasks from using these resources, which interferes with this task performing the specified operation. You can explicitly perform this transition by calling the Control Task function/VI with `Action` set to Reserve. This transition fails if some task resources are currently reserved by another task. If the task can gain access to all the resources it uses, the task is successfully reserved and transitions to the Reserved state. Otherwise, it remains in the Verified state.

Committed State

NI-DAQmx programs some of the settings for the resources when the task is committed. These settings might be the rate of a clock or the input limits of a channel on a device, the direction of a trigger line on a PXI chassis, or the size of the buffer memory in the computer. Other settings, such as the sample counter, cannot be programmed when the task is committed because they need to be programmed every time the task is started. When a task is committed, it transitions from the Reserved state to the Committed state. You can explicitly perform this transition by invoking the Control Task function/VI with `Action` set to Commit. In general, the commit transition should not fail. If it does, it is an exceptional condition and the task remains in the Reserved state. If the settings for the resources used by the task are programmed, the task is successfully committed and transitions to the Committed state.

Running State

When the task begins to perform the specified operation, the task transitions from the Committed state to the Running state. You can explicitly perform this transition by invoking the Start Task function/VI. Notice that starting a task does not necessarily start acquiring samples or generating a waveform. You might have specified the timing and triggering attributes/properties such that a sample is not acquired until you call the Read function/VI or a waveform is not generated until a trigger is detected. In general, the start transition does not fail. If it does, it is an exceptional condition, and the task remains in the Committed state. If the task begins to perform the specified operation, the task is successfully started and transitions to the Running state.

Running to Committed State

The task ceases to perform the specified operation when the task transitions from the Running state to the Committed state. To explicitly perform this transition, call the Stop Task function/VI. Notice that you might have specified the timing and triggering attributes/properties such that all the samples are acquired before this transition occurs. For output operations, the last value written will typically continue to be generated after the task is stopped. In this situation, despite the fact that no additional samples are acquired, the task is still in the Running state until this transition occurs. In general, the stop transition does not fail. If it does, it is an exceptional condition, and the task is returned to the Reserved state. If the task is stopped, the task successfully transitions back to the Committed state.

Committed to Verified State

When the task resources that perform the specified operation are released, the task transitions from the Committed state to the Verified state. These resources may be clocks or channels on a device, trigger lines on a PXI chassis, or buffer memory in the computer. To explicitly perform this transition, call the Control Task function/VI with `Action` set to `Unreserve`. After the task releases all of its resources, it successfully transitions back to the Verified state.

Explicit Versus Implicit State Transitions

In some scenarios, the user performs explicit state transitions. Other times, the user should rely on the task to perform implicit state transitions. Which method you use depends on your application. The following list identifies instances that require explicit state transitions:

- **Verify**—Some applications require users to interactively configure the channel, timing, and triggering attributes/properties of a task. For these applications, explicitly verify the task occasionally to ensure that attribute/property values are valid.
- **Reserve**—If the following are all true, explicitly reserve a task:
 - Your application contains many different tasks that use the same set of resources.
 - One of these tasks performs a repeated operation.
 - You want to ensure that no other tasks acquire the resources used by this task after the repeated operation begins.

Reserving the task exclusively acquires the resources that the task uses. Reserving the task also ensures that other tasks cannot acquire these resources. For example, your application contains two tasks that each perform a sequence of measurements. To ensure that each sequence completes before the other sequence begins, explicitly reserve each task before it begins its sequence of measurements.

- **Commit**—If your application performs multiple measurements/generations by repeatedly starting and stopping a task, explicitly commit a task. Every time the task starts, it must acquire and configure resources. Committing the task exclusively acquires the resources that the task uses and programs some of the settings for these resources. Explicitly committing the task ensures that the task performs these operations once which decreases the overall start time for the task. For example, your application repeatedly performs finite, hardware-timed measurements. Commit the task before repeatedly performing these measurements. This action can dramatically decrease the time that is required to start the task. Explicitly committing a task is also required if you perform additional read operations of these acquired samples. For more information, refer to *Using the Start Task Function/VI*.
- **Start**—If your application repeatedly performs read or write operations, explicitly start a task. Starting the task does the following:

- Reserves the resources used by the task.
- Programs some of the settings for these resources.
- Begins to perform the specified operation.

These operations occur each time the application performs a read or write operation. Explicitly starting the task ensures that the task performs these operations once which considerably decreases the time required. For example, your application repeatedly performs single-sample, software-timed read operations. Explicitly start the task before each read operation. This action can dramatically decrease the time that is required for each read operation.

Related concepts:

- [Using the Start Task function/VI](#)

Implicit Task State Transitions

Although you can explicitly transition a task through each of its states as described in Task State Model, you rarely need this level of detailed control. Two scenarios exist in which a task is implicitly transitioned from one state to another:

- Moving the task through multiple states at the same time
- Operations that require state transitions

Related concepts:

- [Task State Model](#)
- [Task Moves Through Multiple States at the Same Time](#)
- [Operations That Require State Transitions](#)

Task Moves Through Multiple States at the Same Time

Some state transitions require the task to move through one or more states to reach the specified state. For example, if the task is in the Unverified state, and you call the Control Task function/VI, setting `Action` to Reserve, the task is verified and reserved. The task transitions from the Unverified state to the Verified state and to the Reserved state. In most applications, it is not helpful to explicitly transition the task to each state. Instead, invoke only those transitions that are necessary, and the task implicitly handles the rest.

Operations That Require State Transitions

You implicitly transition the task to a new state when you perform an operation that requires that the task be in a specific state and it is not. If this occurs, the task is implicitly transitioned to the required state. Some operations that require state transitions include the following:

- Querying the value of an attribute/property implicitly verifies the task. This verification is required to return accurate coerced values of attributes/properties. Because the coerced value of a attribute/property often depends on the values of other attributes/properties, the task as a whole must be verified to calculate the value. Because the task might be implicitly verified when you query the value of an attribute/property, NI-DAQmx may return an error specifying that the value of attribute/property is invalid.
- Calling the Read function/VI implicitly commits the task if the task is not already committed. If the value of the DAQmx Read `Auto Start` attribute/property is True and the task has not been started, the task also is implicitly started. For more information regarding the auto-start behavior of read operations, refer to [Using the Start Task Function/VI](#).
- Calling the Write function/VI commits the task. If the value of the `Auto-Start` parameter is True, the task also is started. For more information regarding the auto-start behavior of write operations, refer to [Using the Start Task Function/VI](#).

For example, if the task is in the Reserved state, the value of the DAQmx Read `Auto Start` attribute/property is True, and you call the Read function/VI, the task is implicitly committed and started. The task transitions from the Reserved state to the Committed state and to the Running state before the read operation is performed.

In some applications, it is not necessary to explicitly transition the task to any state. Instead, invoke the desired operation and the task implicitly handles everything else.

Related concepts:

- [Using the Start Task function/VI](#)

Transitioning the State Backwards

When a task is implicitly transitioned backwards, it returns to the state of the task prior

to the last operation that resulted in a forward state transition. For example, if the task was in the Verified state and you called the Start Task function/VI to start the task, the task is reserved, committed, and started, transitioning to the Reserved state and to the Committed state before transitioning to the Running state. When you invoke the Stop Task function/VI, the task is not just stopped and transitioned from the Running state to the Committed state. If this were the case, the result is unexpected because the task still has its resources reserved despite the fact that you never explicitly reserved them. Instead, the task is stopped, uncommitted, and unreserved, returning to the Verified state, its state immediately before you performed the last operation that resulted in the state transition, calling the Start Task function/VI.

As another example, suppose the task is in the Reserved state, and you call the Read function/VI to perform a finite measurement. This results in the task implicitly transitioning from the Reserved state to the Committed state and then to the Running state before performing the read operation. When the read operation completes, the task does not remain in the running state. If this were the case, the result is unexpected behavior, because you need to stop the task and unreserve its resources despite the fact you never explicitly reserved the resources or started the task. Instead, after the finite read operation completes, the task is implicitly transitioned from the Running state to the Committed state to the Reserved state. This results in the task returning to the state before you performed the read operation.

Keep in mind that setting the value of a channel, timing, or triggering attribute/property does not implicitly transition the task back to the Unverified state. Instead, the task remains in its current state and is implicitly verified when the next state transition occurs. For example, if the task is in the Reserved state and you set the value of timing attribute/property, the task remains in the Reserved state. The next time the task, either implicitly or explicitly, is committed, the task is verified. Because the task is implicitly verified when the next state transition occurs, NI-DAQmx can return an error specifying that the value of attribute/property is invalid.